

TMA Training Function

Deep Learning with Neural Network

<i>Course</i>	Deep Learning with Neural Network
<i>Trainer</i>	Tuan Le
<i>Designed by</i>	Upskills
<i>Last updated</i>	23-Sep-2024

Objectives

- Learn neural network architectures and apply them for the recognition of objects and text classification tasks using TensorFlow

Contents

- Introduction to Neural Network Architecture
- Feed Forward Neural Networks (FFNNs)
- Convolutional Networks (CNNs)
- Recurrent Neural Network (RNN)

1. Introduction to Neural Network Architecture

- Neural networks are computing systems inspired by the biological neural networks that constitute animal brains.
- It consists of interconnected groups of artificial neurons.
- It recognizes patterns, make decisions, and predict outcomes.

1. Introduction to Neural Network Architecture

■ Key Components of Neural Networks

■ Layers:

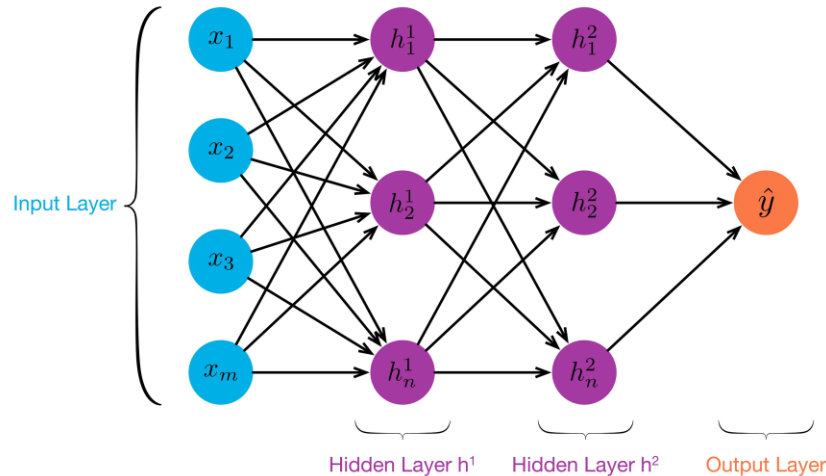
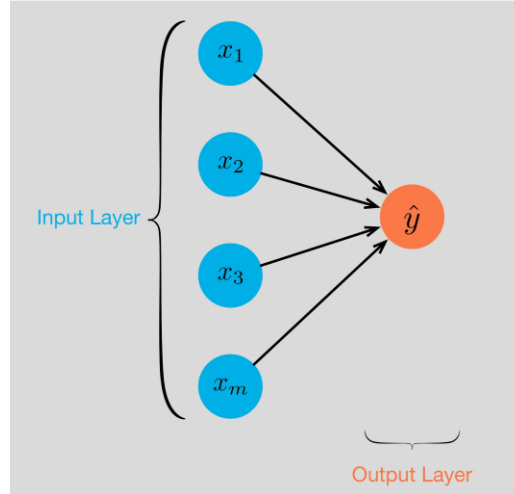
- Input Layer: Takes in the data.
- Hidden Layers: Process the data through weights and activation functions.
- Output Layer: Produces the final result.

- Neurons: Basic units that process inputs and produce outputs.
- Weights and Biases: Parameters that the model learns during training.
- Activation Functions: Introduce non-linearity to the model.

1. Introduction to Neural Network Architecture

■ Layers:

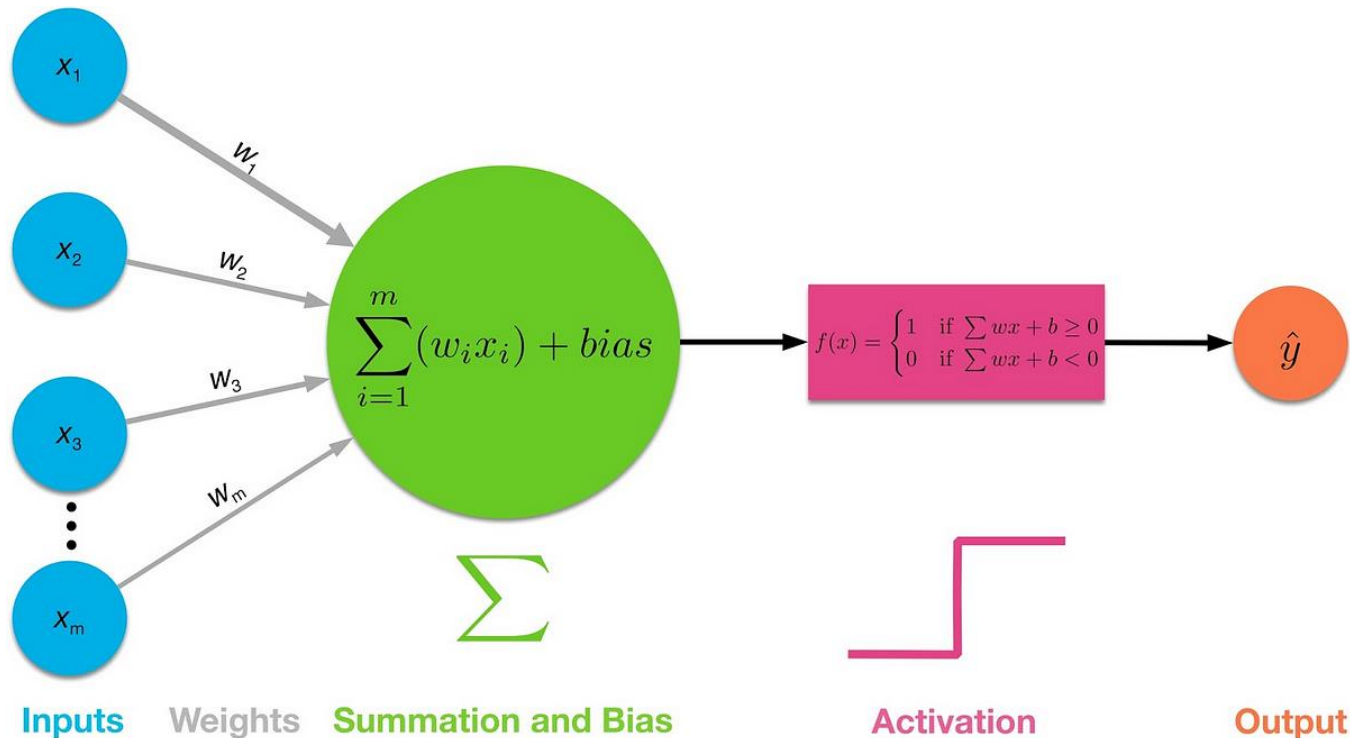
- Input Layer: Takes in the data.
- Hidden Layers: Process the data through weights and activation functions.
- Output Layer: Produces the final result.



1. Introduction to Neural Network Architecture

■ Neurons:

- Receive inputs, apply weights, sum them up, add a bias, and pass through an activation function.

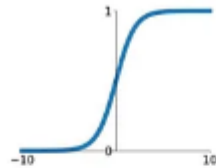


1. Introduction to Neural Network Architecture

■ Activation Functions

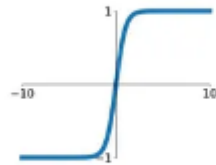
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



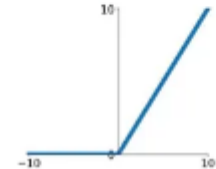
tanh

$$\tanh(x)$$



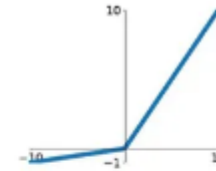
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

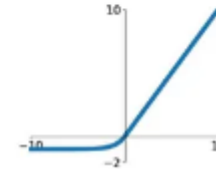


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

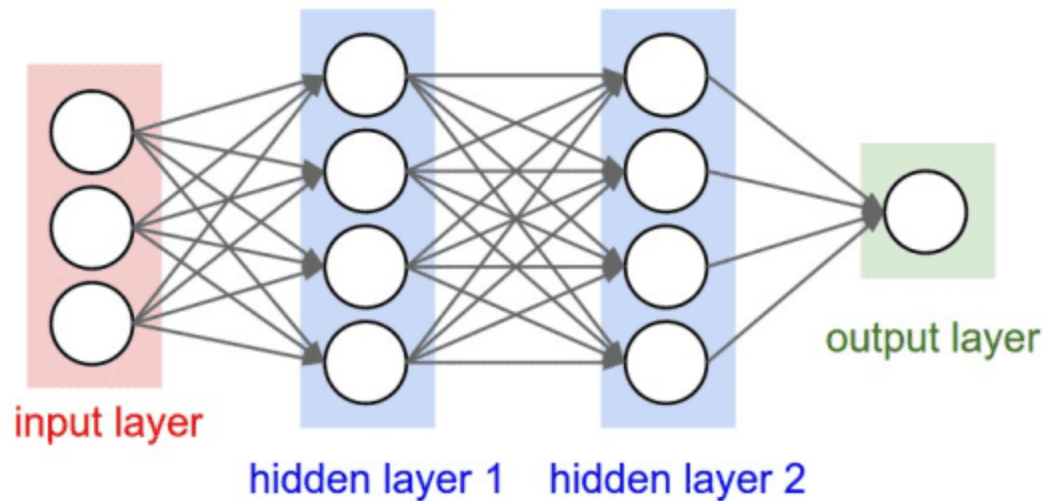
ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



2. Feed Forward Neural Networks

- A type of artificial neural network where connections between the nodes do not form a cycle.
- Data flows in one direction, from input to output

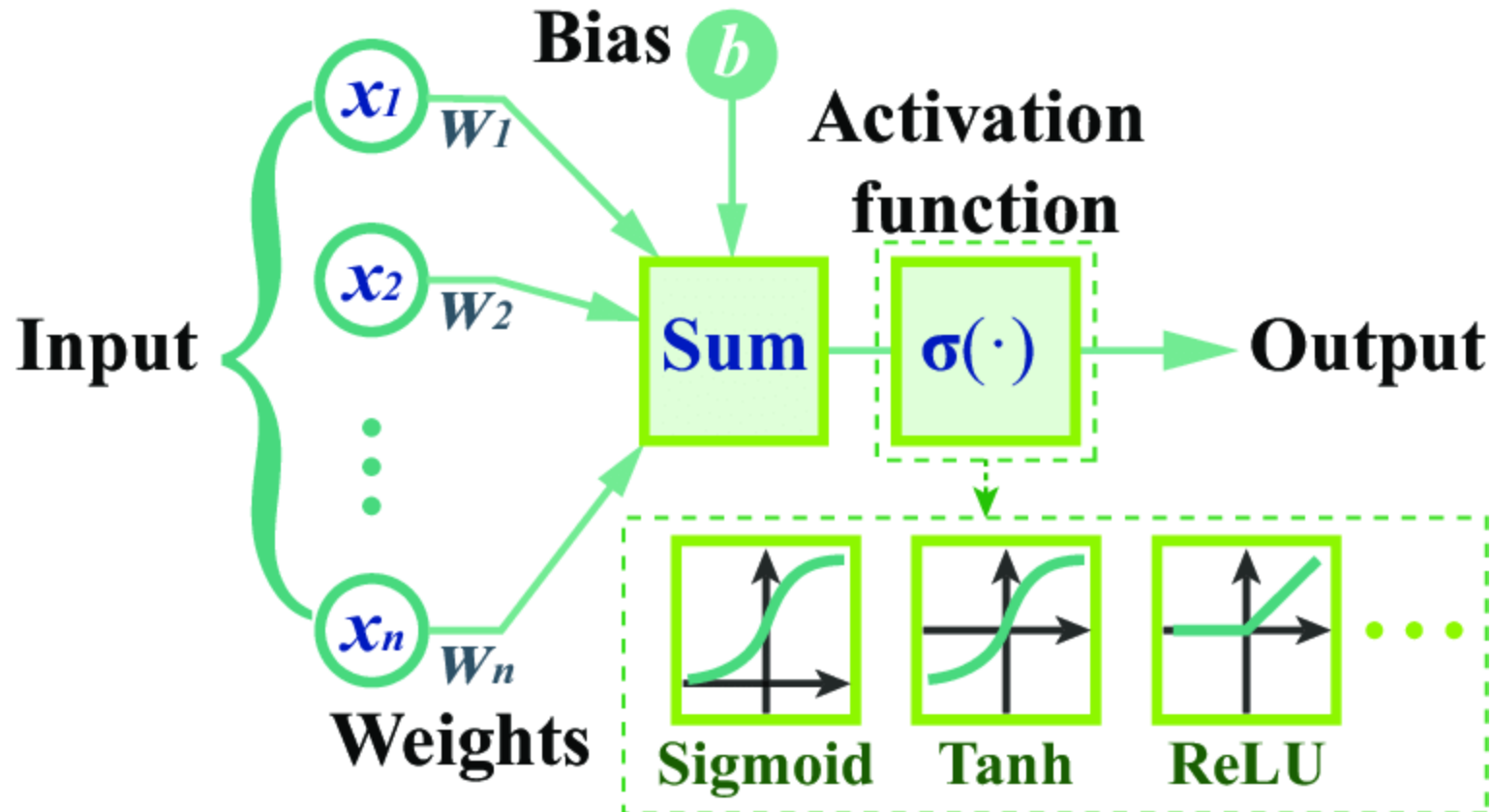


2. Feed Forward Neural Network - Structure

- Layers:
 - Input Layer: Neurons that take the input features.
 - Hidden Layers: One or more layers of neurons that apply transformations.
 - Output Layer: Neurons that produce the final output.
- Neurons: Basic units that perform computations.
- Weights: Parameters that transform the input data within the neurons.
- Biases: Additional parameters in neurons to adjust the output.

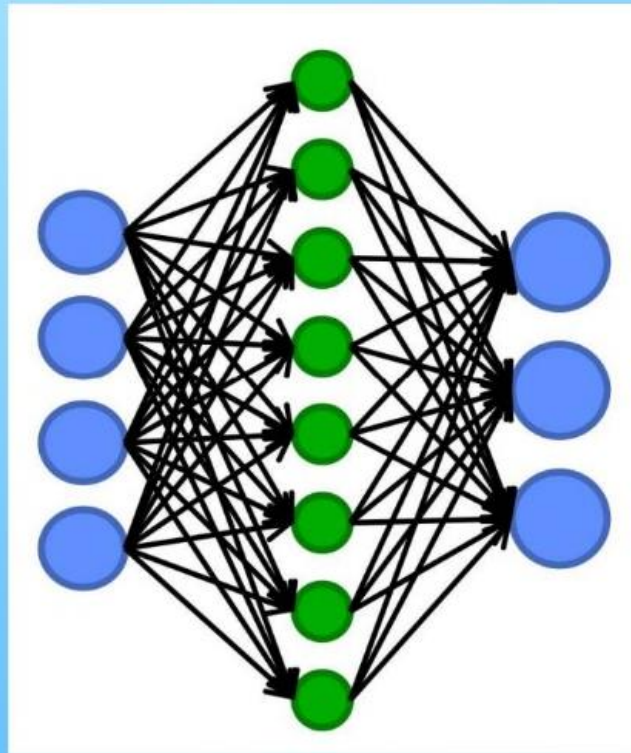
2. Feed Forward Neural Networks

- Forward Propagation
 - Definition: Process of moving inputs through the network to generate the output.



2. Feed Forward Neural Networks

■ Backpropagation



Compute a gradient descent with respect to weights.



Comparing outputs to desired system outputs.

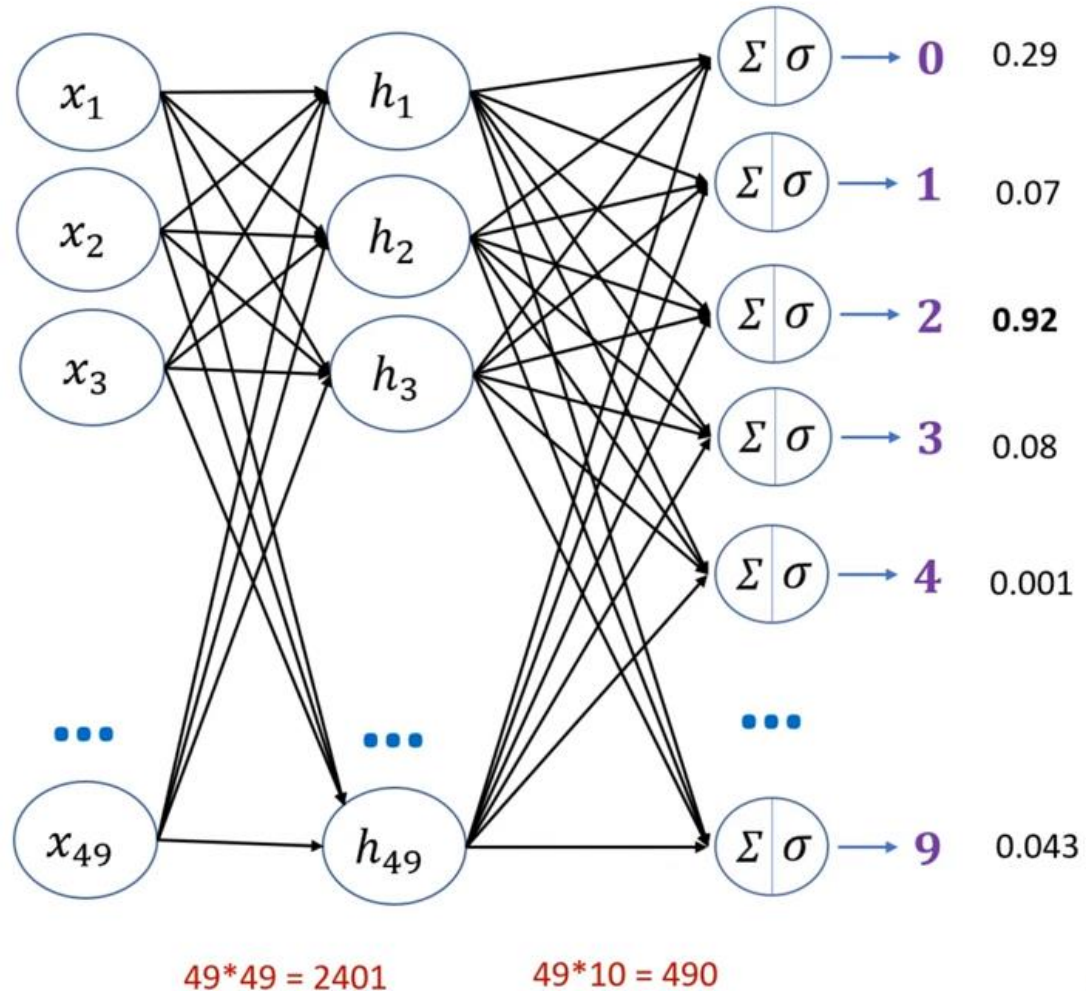
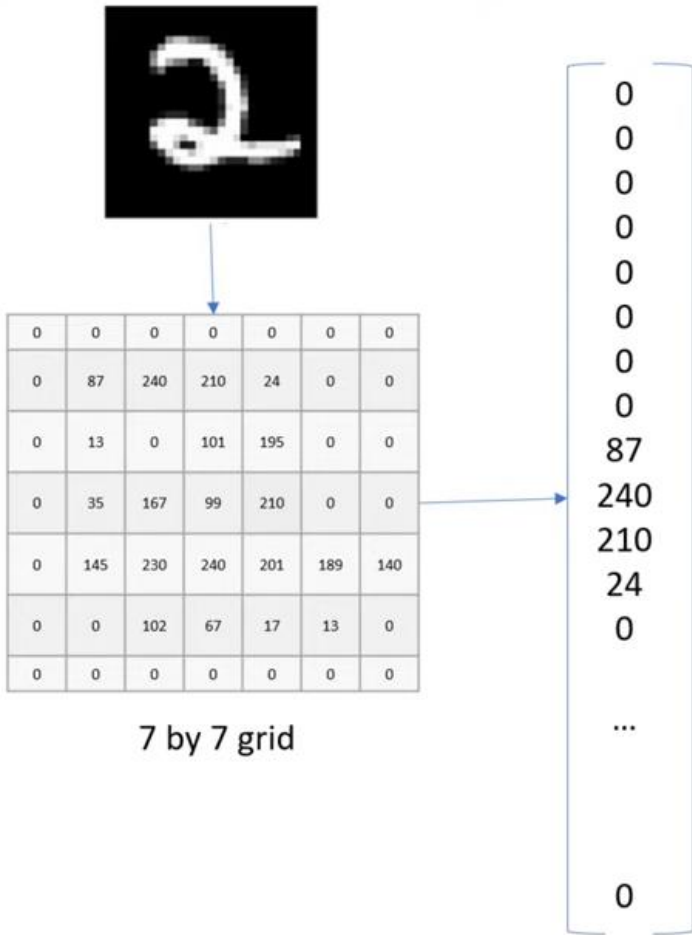


Adjust connection weights to narrow the difference between the two.

2. FNN - Disadvantages

- Poor Feature Learning
- High Parameter Count
- Sensitivity to Input Variations

2. FNN - Disadvantages



3. How CNNs Address FNN Limitations

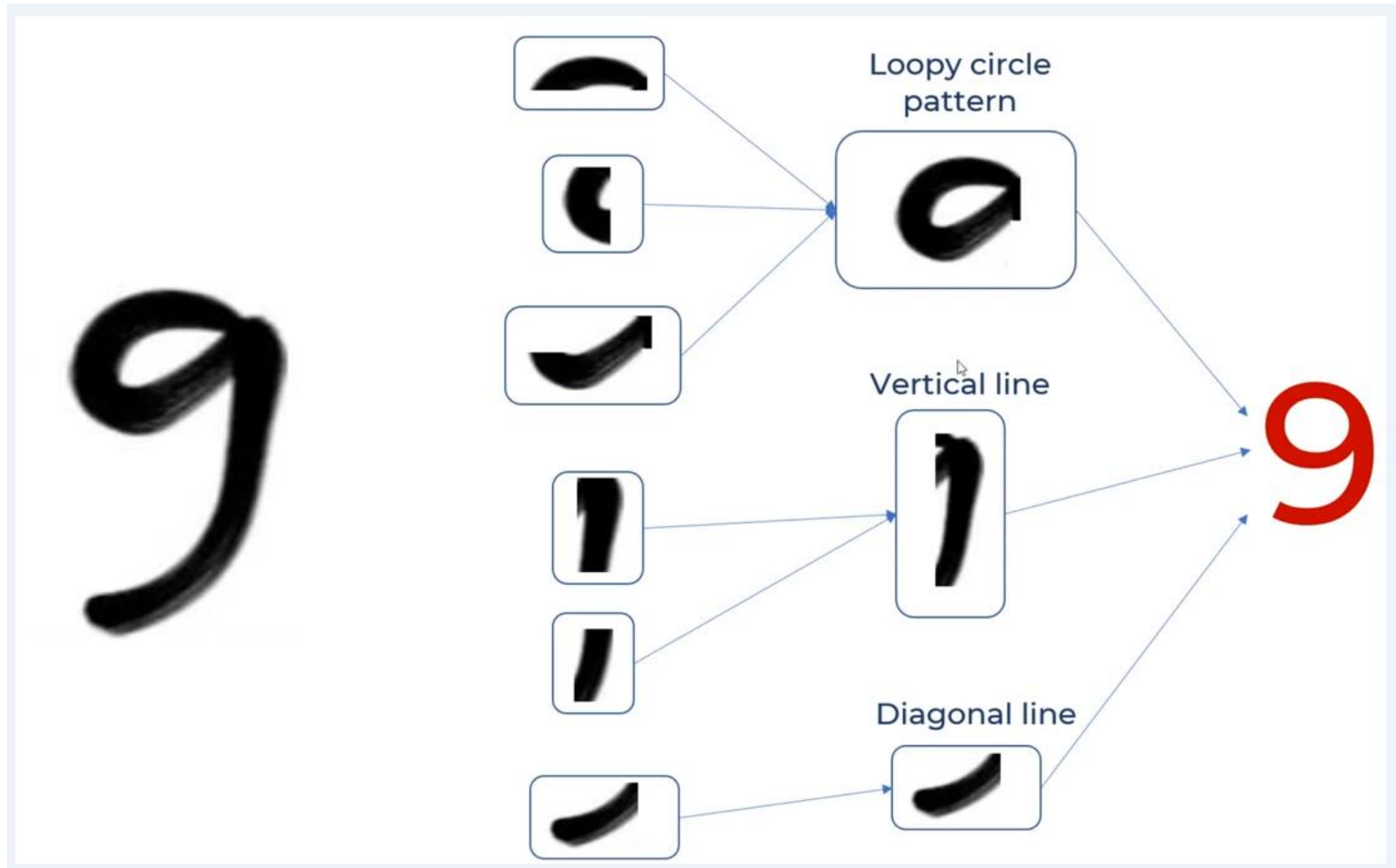
- Automatic Feature Extraction:
 - CNNs automatically learn spatial features without manual intervention.
- Parameter Efficiency:
 - Weight sharing in convolutional layers reduces the number of parameters.
- Hierarchical Learning:
 - Captures local patterns and builds complex features hierarchically.
- Robustness to Distortions:
 - Pooling layers provide translation invariance, enhancing model robustness.

3. Convolutional Networks (CNNs)

- Are a class of deep neural networks, most commonly applied to analyzing visual imagery
- Layers:
 - Convolutional Layers: Apply a convolution operation to the input, passing the result to the next layer.
 - Pooling Layers: Reduce the spatial dimensions of the input volume.
 - Fully Connected Layers: Neurons have full connections to all activations in the previous layer.

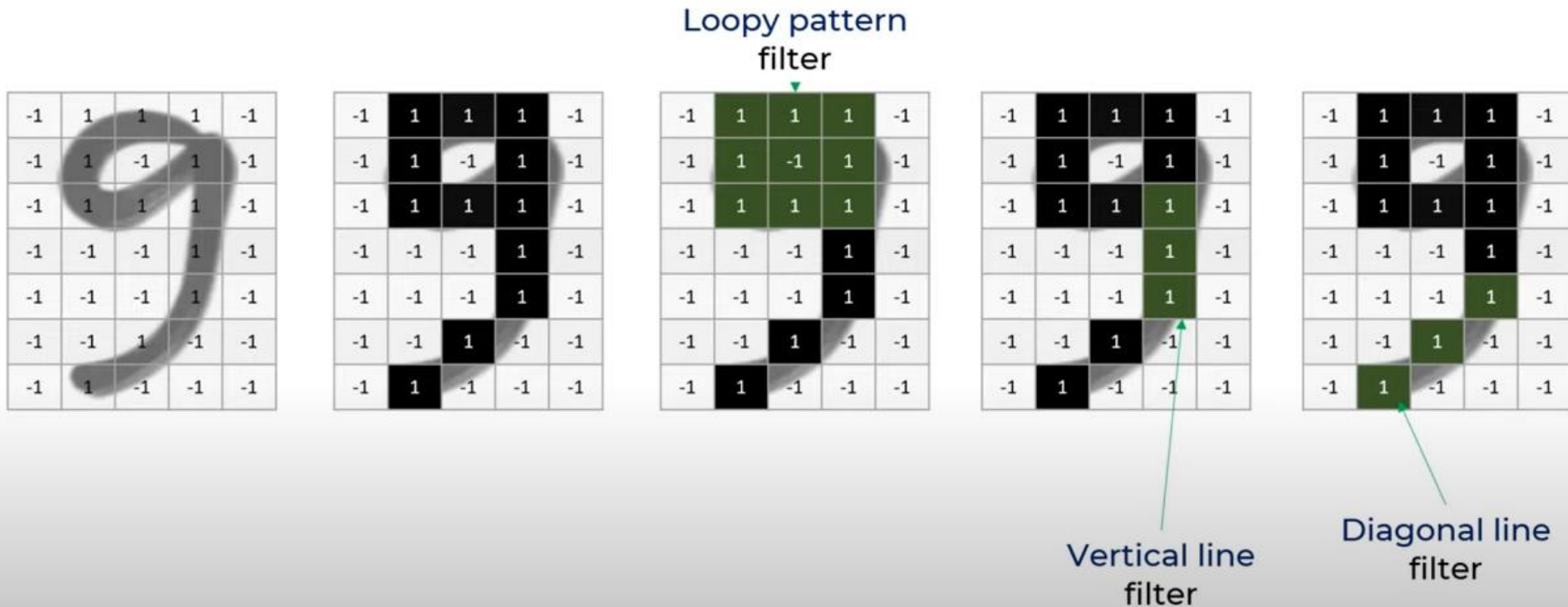
Convolutional Networks (CNNs)

- Convolutional Layer

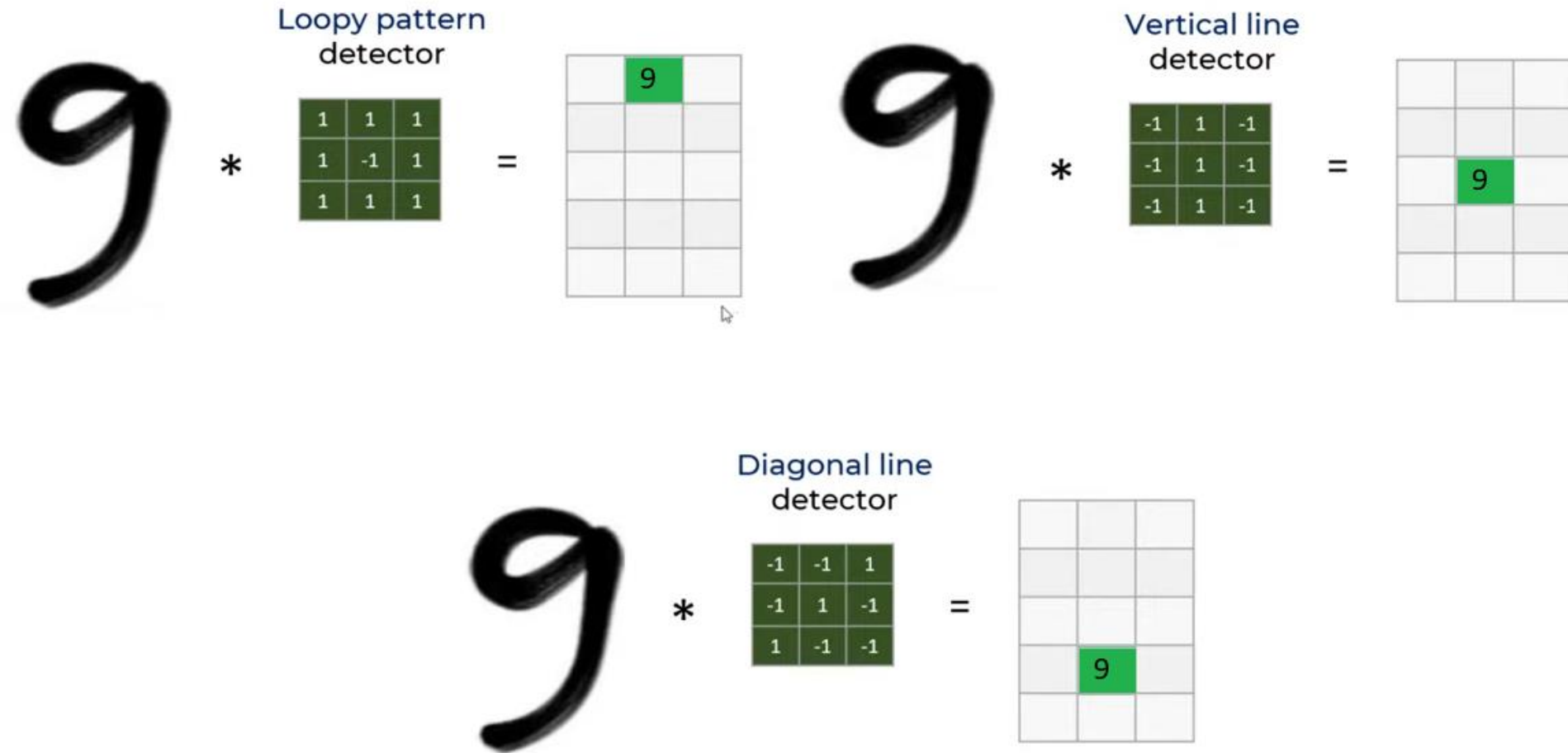


Convolutional Networks (CNNs)

■ Convolutional Layer

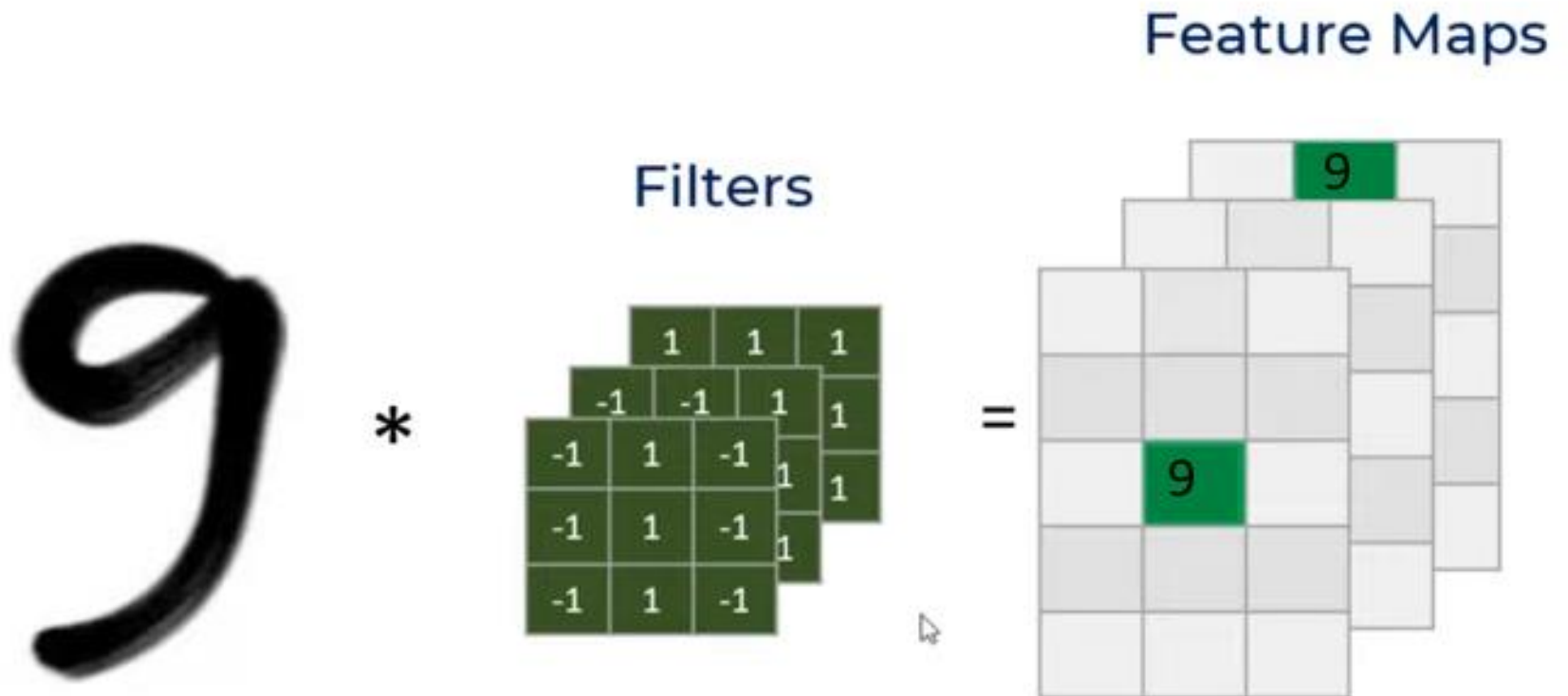


Convolutional Networks (CNNs)



Convolutional Networks (CNNs)

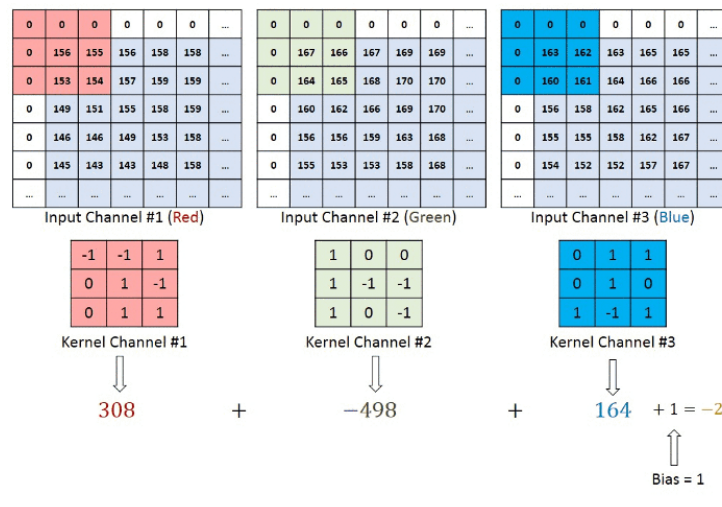
- Convolutional Layer



Convolutional Networks (CNNs)

■ Convolutional Layer

- **Filters (Kernels):** Small matrices used to detect patterns.
- **Stride:** The number of pixels by which the filter matrix moves over the input matrix.
- **Padding:** Adding extra pixels around the edges to control the spatial dimensions of the output.



$$Out_Size = \frac{In_Size - KernelSize + 2 * Padding}{Stride} + 1$$

Output

25				...
				...
				...
				...
...	...	...	...	...

Convolutional Networks (CNNs)

■ Pooling Layer

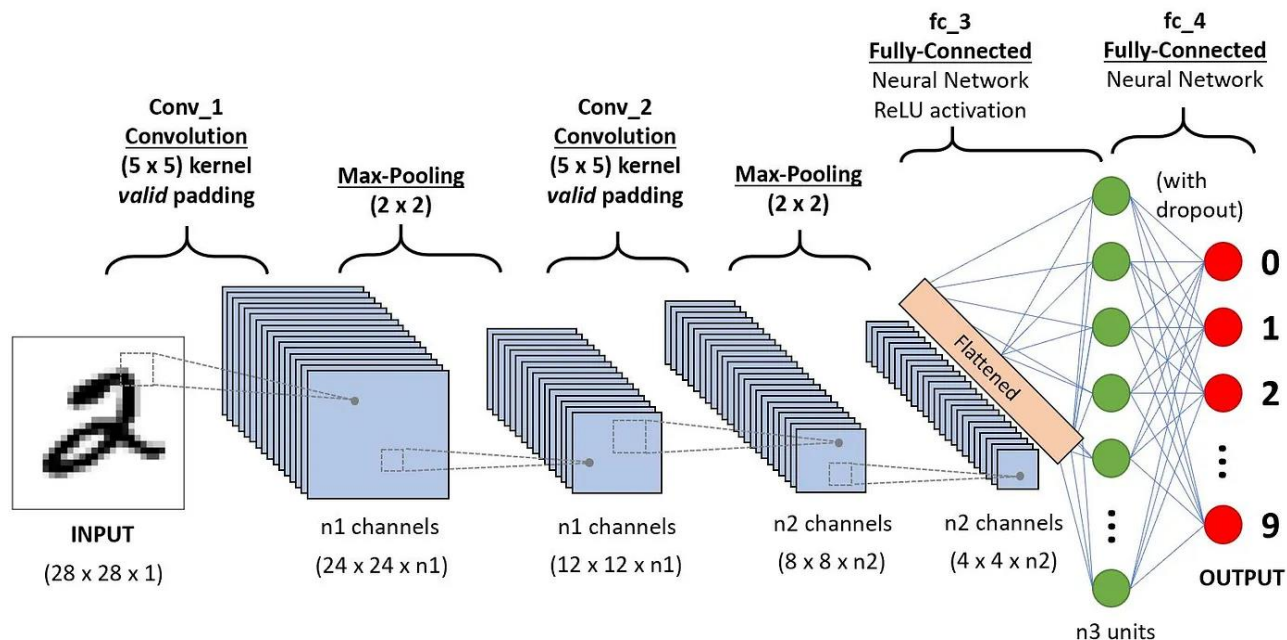
- Purpose: Reduce the spatial dimensions (width and height) of the input volume.
- Types:
 - Max pooling: Takes the maximum value from the window.
 - Average pooling: Takes the average value from the window.

$$Out_Size = \frac{In_Size - PoolSize}{Stride} + 1$$

Convolutional Networks (CNNs)

■ Fully Connected Layer

- **Function:** Connect every neuron in one layer to every neuron in another layer.
- **Role:** Perform classification based on the features extracted by previous layers.



CNNs example - Convolution Operation

- For example : $Out_Size = \frac{In_Size - KernelSize + 2 * Padding}{Stride} + 1$

- $Out_Size = \frac{6 - 3 + 2 * 0}{1} + 1 = 4$

1. Input matrix:

1	2	3	0	1	2
4	5	6	1	0	3
7	8	9	2	4	1
1	2	3	5	6	7
4	5	6	8	9	10
7	8	9	11	12	13

Kernel matrix

1	0	-1
1	0	-1
1	1	1

2. Convolution Output Matrix

20	25	23	3
2	20	25	17
11	22	25	26
20	22	26	32

CNNs example - Convolution Operation

Top-Left:

1	2	3
4	5	6
7	8	9

Kernel matrix

1	0	-1
1	0	-1
1	1	1

$$\text{Convolution}[1.2] = (1 \cdot 1) + (2 \cdot 0) + (3 \cdot -1) + (4 \cdot 1) + (5 \cdot 0) + (6 \cdot -1) + (7 \cdot 1) + (8 \cdot 1) + (9 \cdot 1) = 1 - 3 + 4 - 6 + 7 + 8 + 9 = 20$$

Top-Middle::

2	3	0
5	6	1
8	9	2

$$\text{Convolution}[1.2] = (2 \cdot 1) + (3 \cdot 0) + (0 \cdot -1) + (5 \cdot 1) + (6 \cdot 0) + (1 \cdot -1) + (8 \cdot 1) + (9 \cdot 1) + (2 \cdot 1) = 2 + 5 - 1 + 8 + 9 + 2 = 25$$

Top-Right

3	0	1
6	1	0
9	2	4

$$\text{Convolution}[1.3] = (3 \cdot 1) + (0 \cdot 0) + (1 \cdot -1) + (6 \cdot 1) + (1 \cdot 0) + (0 \cdot -1) + (9 \cdot 1) + (2 \cdot 1) + (4 \cdot 1) = 3 - 1 + 6 + 9 + 2 + 4 = 23$$

Top-Far-Right :

0	1	2
1	0	3
2	4	1

$$\text{Convolution}[1,4] = (0 \cdot 1) + (1 \cdot 0) + (2 \cdot -1) + (1 \cdot 1) + (0 \cdot 0) + (3 \cdot -1) + (2 \cdot 1) + (4 \cdot 1) + (1 \cdot 1) = -2 + 1 - 3 + 2 + 4 + 1 = 3$$

CNNs example - Convolution Operation

Middle-Left

4	5	6
7	8	9
1	2	3

1	0	-1
1	0	-1
1	1	1

$$\text{Convolution}[2,1] = ((4 \cdot 1) + (5 \cdot 0) + (6 \cdot -1) + (7 \cdot 1) + (8 \cdot 0) + (9 \cdot -1) + (1 \cdot 1) + (2 \cdot 1) + (3 \cdot 1)) = 4 - 6 + 7 - 9 + 1 + 2 + 3 = 2$$

Middle-Middle:

5	6	1
8	9	2
2	3	5

$$\text{Convolution}[2,2] = (5 \cdot 1) + (6 \cdot 0) + (1 \cdot -1) + (8 \cdot 1) + (9 \cdot 0) + (2 \cdot -1) + (2 \cdot 1) + (3 \cdot 1) + (5 \cdot 1) = 5 - 1 + 8 - 2 + 2 + 3 + 5 = 20$$

Middle-Right

6	1	0
9	2	4
3	5	6

$$\text{Convolution}[2,3] = (6 \cdot 1) + (1 \cdot 0) + (0 \cdot -1) + (9 \cdot 1) + (2 \cdot 0) + (4 \cdot -1) + (3 \cdot 1) + (5 \cdot 1) + (6 \cdot 1) = 6 + 9 - 4 + 3 + 5 + 6 = 25$$

Middle-Far-Right :

1	0	3
2	4	1
5	6	7

$$\text{Convolution}[2,4] = (1 \cdot 1) + (0 \cdot 0) + (3 \cdot -1) + (2 \cdot 1) + (4 \cdot 0) + (1 \cdot -1) + (5 \cdot 1) + (6 \cdot 1) + (7 \cdot 1) = 1 - 3 + 2 - 1 + 5 + 6 + 7 = 17$$

CNNs example - Convolution Operation

Bottom-Left

7	8	9	1	0	-1
1	2	3	1	0	-1
4	5	6	1	1	1

$$\text{Convolution}[3,1] = (7 \cdot 1) + (8 \cdot 0) + (9 \cdot -1) + (1 \cdot 1) + (2 \cdot 0) + (3 \cdot -1) + (4 \cdot 1) + (5 \cdot 1) + (6 \cdot 1) = 7 - 9 + 1 - 3 + 4 + 5 + 6 = 11$$

Bottom-Middle:

8	9	2
2	3	5
5	6	8

$$\text{Convolution}[3,2] = (8 \cdot 1) + (9 \cdot 0) + (2 \cdot -1) + (2 \cdot 1) + (3 \cdot 0) + (5 \cdot -1) + (5 \cdot 1) + (6 \cdot 1) + (8 \cdot 1) = 8 - 2 + 2 - 5 + 5 + 6 + 8 = 22$$

Bottom-Right

9	2	4
3	5	6
6	8	9

$$\text{Convolution}[3,3] = (9 \cdot 1) + (2 \cdot 0) + (4 \cdot -1) + (3 \cdot 1) + (5 \cdot 0) + (6 \cdot -1) + (6 \cdot 1) + (8 \cdot 1) + (9 \cdot 1) = 9 - 4 + 3 - 6 + 6 + 8 + 9 = 25$$

Bottom-Far-Right :

2	4	1
5	6	7
8	9	10

$$\text{Convolution}[3,4] = (2 \cdot 1) + (4 \cdot 0) + (1 \cdot -1) + (5 \cdot 1) + (6 \cdot 0) + (7 \cdot -1) + (8 \cdot 1) + (9 \cdot 1) + (10 \cdot 1) = 2 - 1 + 5 - 7 + 8 + 9 + 10 = 26$$

CNNs example - Convolution Operation

	1	2	3		1	0	-1
	4	5	6		1	0	-1
Far-Bottom-Left	7	8	9		1	1	1

$$\text{Convolution}[4,1] = (1 \cdot 1) + (2 \cdot 0) + (3 \cdot -1) + (4 \cdot 1) + (5 \cdot 0) + (6 \cdot -1) + (7 \cdot 1) + (8 \cdot 1) + (9 \cdot 1) = 1 - 3 + 4 - 6 + 7 + 8 + 9 = 20$$

	2	3	5
	5	6	8
Far-Bottom-Middle:	8	9	11

$$\text{Convolution}[4,2] = (2 \cdot 1) + (3 \cdot 0) + (5 \cdot -1) + (5 \cdot 1) + (6 \cdot 0) + (8 \cdot -1) + (8 \cdot 1) + (9 \cdot 1) + (11 \cdot 1) = 2 - 5 + 5 - 8 + 8 + 9 + 11 = 22$$

	3	5	6
	6	8	9
Far-Bottom-Right	9	11	12

$$\text{Convolution}[4,3] = (3 \cdot 1) + (5 \cdot 0) + (6 \cdot -1) + (6 \cdot 1) + (8 \cdot 0) + (9 \cdot -1) + (9 \cdot 1) + (11 \cdot 1) + (12 \cdot 1) = 3 - 6 + 6 - 9 + 9 + 11 + 12 = 26$$

	5	6	7
	8	9	10
Far-Bottom-Far-Right :	11	12	13

$$\text{Convolution}[4,4] = (5 \cdot 1) + (6 \cdot 0) + (7 \cdot -1) + (8 \cdot 1) + (9 \cdot 0) + (10 \cdot -1) + (11 \cdot 1) + (12 \cdot 1) + (13 \cdot 1) = 5 - 7 + 8 - 10 + 11 + 12 + 13 = 32$$

CNNs example - Max Pooling Operation

1. Top-Left Region:

20	25
2	20

Max Value: $\max(20, 25, 2, 20) = 25$

2. Top-Middle Region:

25	23
20	25

Max Value: $\max(25, 23, 20, 25) = 25$

3. Top-Right Region:

23	3
25	17

Max Value: $\max(23, 3, 25, 17) = 25$

4. Middle-Left Region:

2	20
11	22

Max Value: $\max(2, 20, 11, 22) = 22$

5. Middle-Middle Region:

20	25
22	25

Max Value: $\max(20, 25, 22, 25) = 25$

6. Middle-Right Region:

25	17
25	26

Max Value: $\max(25, 17, 25, 26) = 26$

7. Bottom-Left Region:

11	22
20	22

Max Value: $\max(11, 22, 20, 22) = 22$

8. Bottom-Middle Region:

22	25
22	26

Max Value: $\max(22, 25, 22, 26) = 26$

CNNs example - Max Pooling Operation

9. Bottom-Right Region:

25	26
26	32

Max Value: $\max(25, 26, 26, 32) = 32$

Max-Pooled Output Matrix (3x3):

25	25	25
22	25	26
22	26	32

CNNs example-Flatten the Max-Pooled

- Step 3: Flatten the Max-Pooled Output
 - [25,25,25,22,25,26,22,26,32]
- Step 4: Fully-Connected Layer with ReLU Activation
 - Assume the weights of the fully connected layer are as follows (3 neurons):

0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9
0.9	0.8	0.7	0.6	0.5	0.4	0.3	0.2	0.1
0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9

- Biases: [1,2,3]

CNNs example-Flatten the Max-Pooled

- Computing the Outputs:

- For the first neuron:

$$0.1 \cdot 25 + 0.2 \cdot 25 + 0.3 \cdot 25 + 0.4 \cdot 22 + 0.5 \cdot 25 + 0.6 \cdot 26 + 0.7 \cdot 22 + 0.8 \cdot 26 + 0.9 \cdot 32 + 1 \\ = 2.5 + 5 + 7.5 + 8.8 + 12.5 + 15.6 + 15.4 + 20.8 + 28.8 + 1 = 118.9$$

- For the second neuron:

$$0.9 \cdot 25 + 0.8 \cdot 25 + 0.7 \cdot 25 + 0.6 \cdot 22 + 0.5 \cdot 25 + 0.4 \cdot 26 + 0.3 \cdot 22 + 0.2 \cdot 26 + 0.1 \cdot 32 + 2 \\ = 22.5 + 20 + 17.5 + 13.2 + 12.5 + 10.4 + 6.6 + 5.2 + 3.2 + 2 = 113.1$$

- For the third neuron:

$$0.1 \cdot 25 + 0.2 \cdot 25 + 0.3 \cdot 25 + 0.4 \cdot 22 + 0.5 \cdot 25 + 0.6 \cdot 26 + 0.7 \cdot 22 + 0.8 \cdot 26 + 0.9 \cdot 32 + 3 \\ = 2.5 + 5 + 7.5 + 8.8 + 12.5 + 15.6 + 15.4 + 20.8 + 28.8 + 3 = 120.9$$

- Final Outputs:

- $[118.9, 113.1, 120.9]$

Limitations of CNN

- **Static Nature:**

- CNNs are designed for fixed-size inputs and do not inherently handle sequence information.

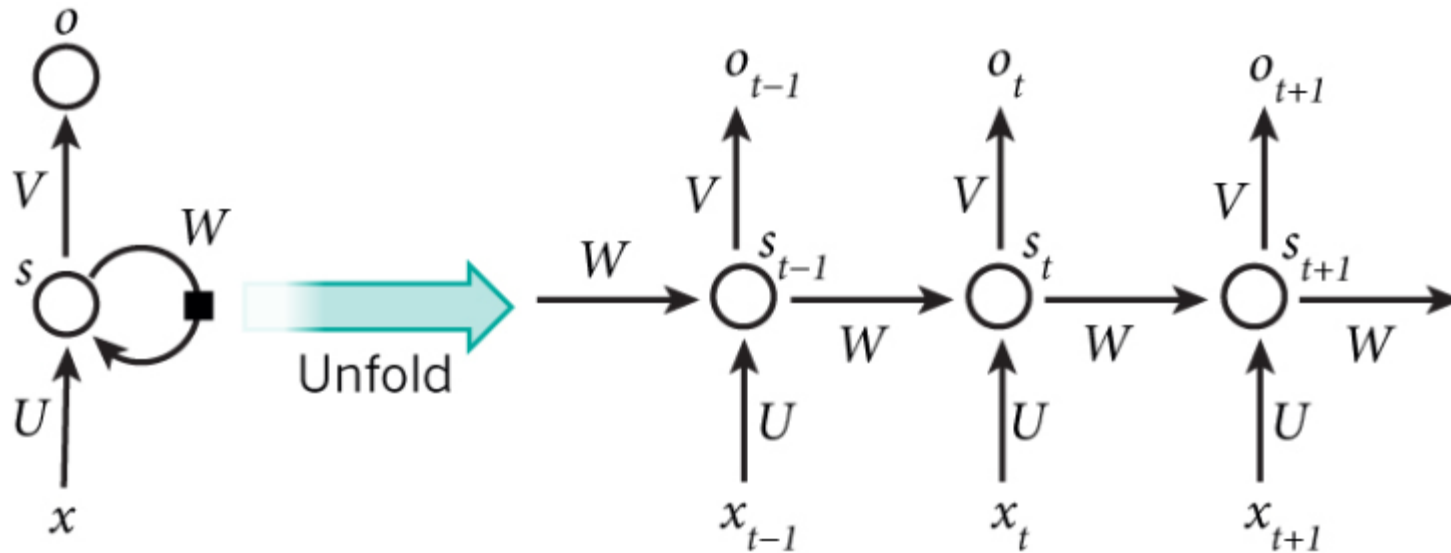
- **Loss of Spatial/Temporal Context:**

- CNNs may not effectively leverage relationships in sequential data due to their local receptive fields.

4. Recurrent Neural Network (RNN)

- Neural networks designed to work with sequential data
- Ability to maintain internal state (memory)
- Applications: Natural language processing, time series analysis, speech recognition

4. RNN architecture



Mathematical Formulation of RNNs

■ Hidden State Update:

- $s_t = f(U * x_t + W * s_{t-1} + b_s)$
- s_t is the hidden state at time step t
- x_t is the input at time step t
- U is the weight matrix for input-to-hidden connections
- W is the weight matrix for hidden-to-hidden connections
- b_s is the bias vector for the hidden state
- f is an activation function (typically tanh or ReLU)

Mathematical Formulation of RNNs

■ Output Calculation:

- $o_t = g(V^*s_{t-1} + b_o)$
- o_t is the output at time step t
- V is the weight matrix for hidden-to-output connections
- b_o is the bias vector for the output
- g is an activation function (often softmax for classification tasks)

Mathematical Formulation of RNNs

- Training Objective: The network is typically trained to minimize a loss function L:
 - $L = \sum_t L_t(y_t, o_t)$
 - L_t is the loss at time step t
 - y_t is the true target at time step t
 - o_t is the predicted output at time step t
- Parameter Updates:
 - During training, the parameters U, W, V, b_s , and b_o are updated using gradient descent:
 - $\theta = \theta - \alpha * \frac{\partial L}{\partial \theta}$
 - θ represents any of the parameters α is the learning rate
 - $\frac{\partial L}{\partial \theta}$ is the gradient of the loss with respect to the parameter

Mathematical Formulation of RNNs

- Backpropagation Through Time (BPTT):

- *We have:*

- $L = \sum_t L_t(y_t, o_t)$

- $s_t = f(U^*x_t + W * s_{t-1} + b_s)$

- $o_t = g(V^*s_{t-1} + b_o)$

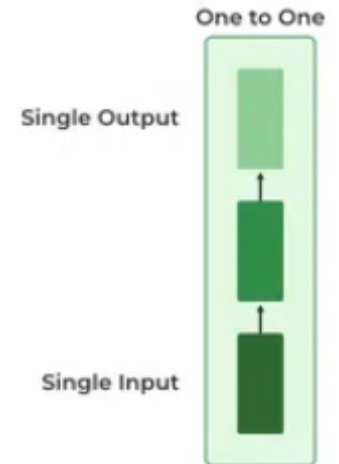
- The gradients are computed by unrolling the network and applying the chain rule:

- $\frac{\partial L}{\partial \theta} = \sum_t \frac{\partial L_t}{\partial o_t} * \frac{\partial o_t}{\partial s_t} * \frac{\partial s_t}{\partial \theta}$

Types of RNNs

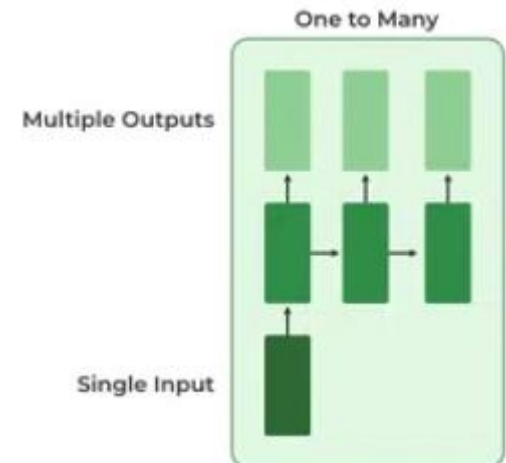
■ One-to-One

- Each input is mapped to a single output.
- It is essentially a traditional neural network without recurrence, meaning it doesn't leverage sequential information.
- Suitable for tasks like image classification



■ One-to-Many

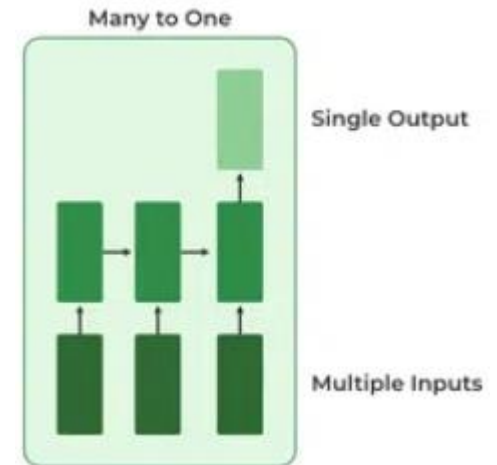
- A single input is used to generate a sequence of outputs.
- It is useful when one input triggers a series of events.
- Used for tasks like image captioning.



Types of RNNs

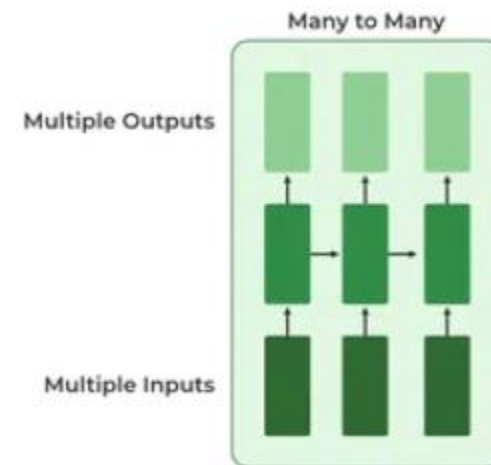
■ Many-to-One

- A sequence of inputs is used to produce a single output.
- It is ideal for tasks where the output depends on the entire input sequence.
- Ideal for tasks like sentiment analysis.



■ Many-to-Many

- A sequence of inputs is used to produce a sequence of outputs.
- It is versatile and can handle tasks where both inputs and outputs are sequences.
- Applicable to tasks like machine translation.



Applications of RNNs

- Language Modelling and Generating Text
- Machine Translation
- Speech Recognition
- Generating Image Descriptions
- Text Summarization
- Call Center Analysis

RNN case study - Text classification

